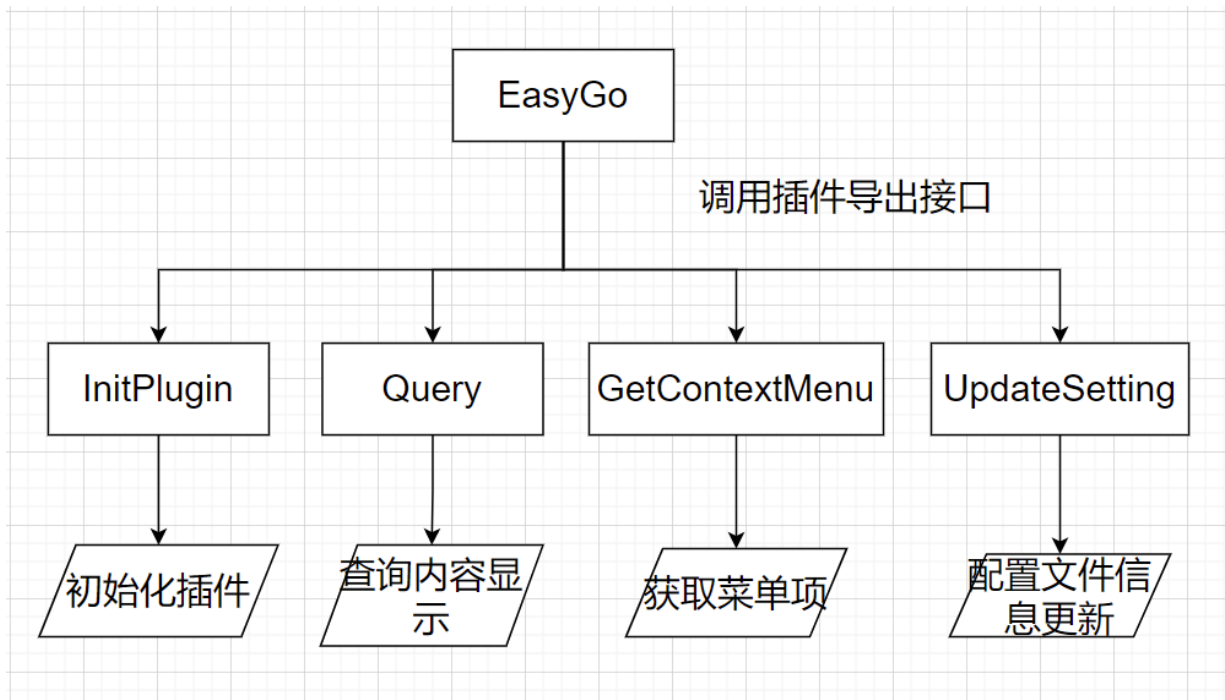


简介

EasyGo借鉴了wox的插件架构，尤其是python插件，几乎差不多，所以如果你开发过wox的python插件，那么可以很容易的将它移植到EasyGo上面来，目前易语言的插件还没经过大量测试，欢迎来到群里交流探讨

架构

插件的整体架构如图



插件接口介绍

插件导出三个基本接口函数供EasyGo调用，它和c++和python的有所区别，大概介绍如下：

1. InitPlugin

子程序名	返回值类型	公开	易包	备 注	
InitPlugin	逻辑型	✓		初始化插件，插件可以在这里做一些初始化操作，只会调用一次	
参数名	类 型	参 考	可 空	数 组	备 注
pluginPath	文本型				当前插件位于的文件夹路径，结尾不带斜杠

```
api.初始化 (pluginPath)
```

```
return (真)
```

2. Query

子程序名	返回值类型	公开	易包	备 注	
Query	文本型	✓		查询函数，使用EasyGo传入的查询关键字来获取结果列表并返回显示	
参数名	类 型	参考	可空	数组	备 注
查询内容	文本型				查询的json文本，可以转换为“查询内容”数据结构
变量名	类 型	静态	数组	备 注	
result	查询结果				
query	查询内容				
resultList	查询结果		0		

apihelper.解析查询内容 (查询内容, query)

```
if (query.参数 = "")
    result.标题 = "易语言测试插件"
    result.内容 = "这是一个易语言插件模板"
    ' 这个iconPath是相对路径，位于该插件目录，可以不填，默认插件配置文件指定的图标
    result.图标路径 = "app.png"
    AddElement (resultList, result)
return (apihelper.生成查询结果 (resultList))

result.标题 = "易语言测试插件"
result.内容 = query.参数
AddElement (resultList, result)

return (apihelper.生成查询结果 (resultList))
```

3. GetContextMenu

子程序名	返回值类型	公开	易包	备 注	
GetContextMenu	文本型	✓		获取针对某条结果的菜单选项，一般用不上，我们返回空即可	
参数名	类 型	参考	可空	数组	备 注
查询结果	文本型				被选中的结果项，为“查询结果”数据类型

```
return ("")
```

4. UpdateSetting

子程序名	返回值类型	公开	易包	备 注	
UpdateSetting		✓		当用户进行插件设置后会调用该函数通知更新配置内容	

数据结构介绍

接着我们来说明一下这三个函数涉及到的json字符串结构

文本型 Query(文本型 查询内容)

查询内容

```
1 {
2     "RawQuery": "lm xxxx",
3     "Keyword": "lm",
```

```
4     "Parameter": "xxxx"
5 }
```

它可以转化为下面的数据类型来理解

数据类型名	公开	备注		
查询内容	✓	调用Query查询的时候传入的内容		
成员名	类型	传址	数组	备注
原始文本	文本型			原始字符串, 例如: lm xxx
关键字	文本型			关键字, 例如: lm
参数	文本型			参数, 例如: xxx

返回的结果:

```
1 {
2     "Results": [
3         {
4             "Title": "结果1",
5             "ShowType": 0,
6             "SubTitle": "这是结果1",
7             "IconPath": "app.png",
8             "Action": {
9                 "FuncName": "Ra_CopyPath",
10                "Parameter": "参数",
11                "HideWindow": true
12            }
13        },
14        {
15            "Title": "结果2",
16            "ShowType": 0,
17            "SubTitle": "这是结果2",
18            "IconPath": "app.png",
19            "Action": {
20                "FuncName": "Ra_CopyPath",
21                "Parameter": "参数",
22                "HideWindow": true
23            }
24        }
25    ]
26 }
```

```
26 }
```

它可以转化成如下的数据类型来理解

数据类型名	公开	备注		
响应函数	✓	用于响应结果项被触发时执行的操作		
成员名	类型	传址	数组	备注
函数名	文本型			函数名，如果是指向你自己的函数，则需要将其公开，函数返回为空，只有一个参数
参数	文本型			函数的参数，如果你想传入多个参数，可以考虑传入json字符串，然后自己解析
是否隐藏窗口	逻辑型			执行完毕后是否隐藏 EasyGo 的查询窗口

数据类型名	公开	备注		
查询结果	✓	结果项		
成员名	类型	传址	数组	备注
插件ID	文本型			插件唯一识别guid，不需要赋值，由EasyGo赋值
显示类型	整型			显示类型，0：普通列表项，1：文本框，适合长文本显示
标题	文本型			结果项上行文字，showType为1的时候，为文本标题
内容	文本型			结果项下行文字，showType为1的时候，为文本内容
图标路径	文本型			结果项显示使用的图标路径，支持相对路径，相对于插件目录。为空的时候默认使用插件配置的默认图标
用户数据	文本型			用户自定义数据，会在调用GetContextMenu的时候传入
动作	响应函数			结果项响应函数

文本型 GetContextMenu(文本型 查询结果)

查询结果，这里的查询结果只传入了如下四个字段

```
1 {
2     "ID": "6064551FD7904BD09D23FB9CA9F4CD03",
3     "Title": "结果1",
4     "SubTitle": "这是结果1",
5     "ExtraData": "这个是可选的，由Query中赋值得来"
6 }
```

返回的菜单json，和Query返回的格式一样，是json数组

EasyGo提供给插件的接口

EasyGo提供给插件的有两种类型接口，主动的和被动的，大部分是被动调用，涉及到 PluginAction 结构体。


```

4  "Argc":1,                                这个是代表插件参数个数，关键字+子关键字
5  "Name":"生成Guid",                      插件名
6  "Description":"生成Guid",                插件描述
7  "Author":"zsxxxx",                      插件作者
8  "Version":"1.0.0",                      插件版本
9  "PluginType":"python",                  插件类型，目前有c++，python和e三种
10 "PluginMode":"EnterMode",                插件模式，EnterMode或者RealMode，此字段可以有
11 "AcceptType": "png,ico,folder",          插件支持的拖入的文件类型，就是文件的后缀名，
12 "EnableSeparate": 1,                    是否开启插件分离窗口的支持，默认为1开启
13 "IconPath": "img\\app.png",              插件图标
14 "ExeName":"Main.py" ,                  插件源文件名字
15 "CfgPath": "set.json"                  插件的配置选项，使用相对路径，指定json配置文件路径。可
16 }

```

插件CfgPath配置文件

配置文件json，配置项均为文本格式，而且不支持多级，类似map结构

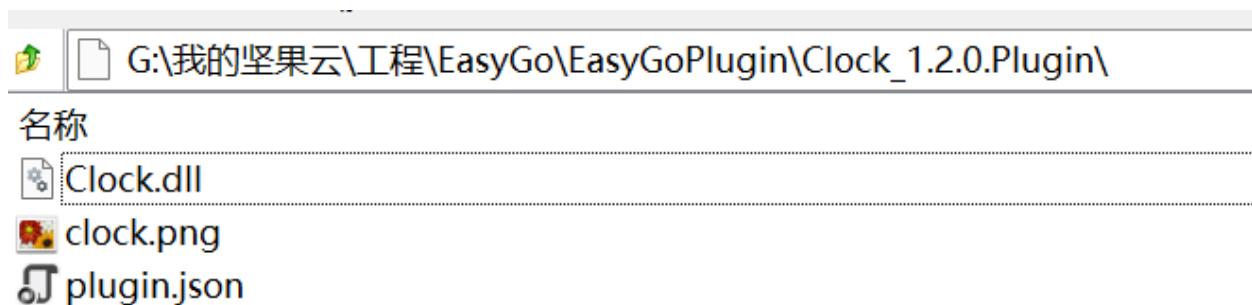
```

1 {
2     "name": "admin",
3     "password": "admin",
4     "role": "admin"
5 }

```

插件打包

插件打包后的格式是 `插件名.Plugin`，其实就是将插件的三个文件打包成zip文件，然后把zip后缀改成plugin即可，记住了，打包的时候插件的外层不要有文件夹



如果你有说明文件，那么请加入 `readme.txt` 文件，全小写，然后加入帮助内容，用户在插件管理页面可以通过右键详情打开帮助文档，然后你也可以放入手动配置文件，在

plugin.json指定（见上），然后用户可以在插件管理页面通过右键设置打开配置文件进行配置

具体的使用方法请参考示例